

DPO on a T4 with Layer Streaming: Take-Home Report

Model: meta-llama/Llama-3.1-8B-Instruct | QLoRA 4-bit NF4 | LoRA r=8, alpha=16, target q_proj+v_proj | max_length=256 (max_prompt_length=128) | batch 1 x accum 4, lr 5e-5, fp16 | Colab T4 (15 GB usable, no bf16) | ~500 Russian preference pairs | plain TRL DPOTrainer fallback script | GB = 10⁹ bytes

VERDICT: DON'T SHIP. The adapter weights demonstrably moved (lora_B L2 norm = 1.998), but I never obtained a healthy end-to-end run through `soup_train`, the observed 6.63 GiB peak is 35% below my worst-case budget and is not yet explained (possibly silent gradient checkpointing, unchecked), and there is no evidence that the model got better or did not get worse.

1. Memory budget (theoretical vs. observed)

Llama-3.1-8B: 8.03B params, hidden 4096, 32 layers, MLP 14336, GQA (8 KV heads), vocab 128,256, untied embedding/lm_head. Budget is for the fallback script as submitted: `device_map={'':0}` (all layers resident, **no layer streaming**), no gradient checkpointing (DPOConfig default is off), fp16, one pair per step = 2 sequences x 256 = 512 tokens (worst case), reference = adapter disabled.

Consumer	Calculation	GB
NF4 linear weights	32 layers x 218.1M = 6.98B params x 0.5 B	3.49
Quantization state	absmax, block 64, double-quant on: 6.98B x 0.0158 B (0.44 GB without it)	0.11
Embedding + lm_head (fp16)	2 x 128,256 x 4,096 x 2 B. Not quantized. dtype is not set in the script; transformers overrides None to fp16 for bnb (to confirm in the load log; fp32 would be 4.20)	2.10
LoRA r=8 (q_proj, v_proj) + grads + Adam	q: 8 x (4096+4096), v: 8 x (4096+1024) = 106,496/layer x 32 = 3.41M params x 16 B (fp32 w + grad + m + v)	0.05
Activations, no checkpointing	~33 x h fp16-values per token per layer (SwiGLU 3.5h intermediates, fp32 RMSNorm copies, GQA-expanded K/V) = 138 MB/layer at 512 tokens = 4.4 GB upper count; shared tensors reduce it, I take ~3.5 GB (+/-30%)	3.50
Dequantization temp	largest layer weight dequantized to fp16 (MLP 58.7M x 2 B)	0.12
Policy logits + loss	512 x 128,256 = 65.7M elements: fp16 0.13 + fp32 upcast 0.26 + log_softmax 0.26 + grad 0.26	0.90
DPO reference pass	No second model (adapter disabled = 0 GB weights); no_grad forward while the policy graph is still alive: transient logits, upper bound	0.50
CUDA context + workspaces	0.30 (invisible to torch allocator, visible to nvidia-smi) + 0.20 cuBLAS/bnb	0.50
Expected total	Worst case (512 tokens), before allocator fragmentation (+5-15%)	11.27

Observed: `torch.cuda.max_memory_allocated()` = **6.63 GiB = 7.12 GB** (the script divides by 1024³; process-wide peak, counter not reset before training, read after training). PyTorch's counter excludes the CUDA context (0.30 GB), so the comparable budget is 11.27 - 0.30 = **10.97 GB**; the observation is **3.85 GB (35%) below** it. The direction matters: a peak *below* a worst-case budget is fine only if I can show why. The loaded-but-idle allocated footprint is 3.49 + 0.11 + 2.10 + 0.05 + 0.20 = 5.95 GB, so training added just 1.17 GB on top. Excluding the 0.12 GB dequantization temp, activations + logits + reference cost ~9.6 MB per token (4.9 GB / 512), so 1.05 GB corresponds to ~110 tokens per pair (~55 per sequence) instead of 512, which is what the data implies (avg 2,826 chars per row, see section 3). Explanations:

- **Short sequences (now unlikely).** The data inspection shows avg 2,826 chars (max 19,793) per row, so most pairs should hit the 128 + 128 token caps, i.e. ~512 tokens per pair: the worst case is the expected case.
- **Gradient checkpointing possibly on (most likely, unchecked).** I assumed it is off (the TrainingArguments default), but some recent TRL versions enable it by default in the config, and the script never prints it. With it, activations drop from 3.5 to ~0.3 GB and the allocated-only budget becomes 11.27 - 3.5 + 0.3 - 0.3 = 7.77 GB = 7.24 GiB, only 0.61 GiB (8%) above the observed 6.63 GiB: within my estimation error. Check: `print(dpo_trainer.args.gradient_checkpointing)`.
- **Different run.** If 6.63 GiB came from another process (e.g. the stalled `soup_train` with layer streaming, where only a window of layers is resident and no full backward ran), this budget does not apply to it.
- **Measurement scope.** The counter is a true peak, so sampling is not the issue, but it was not reset, so it also contains model-loading transients (this can only overstate). `nvidia-smi` is a post-hoc snapshot of reserved memory plus context, not a peak; it should read above 7.1 GB. If the raw file shows less, one of the two measurements is not what I think it is.

Still needed: the gradient-checkpointing flag, `torch.cuda.reset_peak_memory_stats()` right before `train()`, and logged token lengths per batch. Memory was never the blocker: the 11.3 GB worst case fits on the T4 (15 GB); gradient checkpointing would cut activations from ~3.5 GB to ~0.3 GB if longer sequences are needed.

2. Proof of training

Why falling loss proves nothing. DPO loss = $-\log \text{sigmoid}(\beta \times ((\pi_c - \text{ref}_c) - (\pi_r - \text{ref}_r)))$. It can fall because the model fits artifacts of the pairs (length, greiving templates) rather than quality; it is noisy per batch (batch size 1, accum 4); and it is meaningless if the reference log-probs are wrong. If the adapter is not disabled for the reference pass, policy = reference and the loss sits at $\ln 2 = 0.693$ by construction. A run can also finish, log a loss and print SHIP with a frozen, unattached or never-saved adapter.

Verification check. Right after `get_peft_model` and before training, the script snapshots every parameter whose name contains "lora" (A and B) to CPU (`initial_lora_weights`). After `train()` it takes the same snapshot, and for each tensor computes $\|\text{theta_after} - \text{theta_before}\|_2$ in fp32; the per-tensor norms are combined as $\sqrt{\text{sum of squares}}$, i.e. the global L2 norm of the change. LoRA initialises B = 0 (A random), so $dW = B A$ is exactly zero at step 0; a non-zero difference proves the adapter was attached, received gradients, was stepped by the optimizer, and that the tensors inspected are the live trained ones. The script prints PASS if the norm exceeds 1e-5. **Result: L2 norm = 1.998** (raw output attached), five orders of magnitude above the threshold. Note `save_strategy="no"`: nothing is written to disk, so the check runs in memory and the file that would be shipped is not covered. Sanity check: 500 pairs / (batch 1 x accum 4) = ~125 optimizer steps at lr 5e-5 with default linear decay, so one element can move at most $\sim 125 \times 5e-5 / 2 = 3.1e-3$ (Adam step ~ lr). With 3.41M LoRA elements (A 2.10M + B 1.31M) the RMS shift is $1.998 / \sqrt{(3.41M)} = 1.1e-3$, about a third of that ceiling: non-zero, plausible, not saturated.

What this check does NOT detect. Catastrophic forgetting: the norm measures how far the adapter moved, not what was lost. The model may have lost general language skills (Russian fluency outside support tickets, instruction following, Llama 3.1's safety behaviour) and the norm would look identical or better. This needs a before/after evaluation on neutral held-out Russian text and a general benchmark. **Reward hacking:** DPO can raise its implicit reward margin by exploiting patterns in the chosen answers (longer replies, template politeness, hedging or refusal phrasing) without answering better; in my data 239 of 500 rejected answers open with "Konechno..." ("Sure...") versus 2 of 500 chosen, and rejected answers are 1.5x longer, so avoiding the opener and being terse is an easy way to win; a weight norm cannot separate this from real improvement. This needs held-out preference accuracy, length-controlled comparison and reading generations. **Also missed:** non-finite weights (an inf norm would pass the 1e-5 test; only NaN fails it); which of A or B moved (they are summed); the direction and quality of the change (1.998

fits learning as well as noise or a too-high learning rate); whether the change comes from the preference signal at all (a shuffled-label control run would probably give a similar norm); and whether inference-time tokenization matches training.

3. Silent failures

Every row below rests on something I observed: two failures during my runs, the Soup dataset inspection of `ru_dpo_500.jsonl` with my own counts over all 500 rows, and my fallback script. Risks I did not test (fp16 skipped steps, the reference pass, BOS/pad handling, token-level truncation) are not claimed as findings; they appear in section 4 as checks to run. The fallback script bypasses Soup, so pre-flight and soup ship never saw it, and I make no claim about whether they would catch these.

Failure	What could go wrong	Check / evidence	Soup catches?
1. soup train stalls (observed)	Process alive, model resident in VRAM, but step counter and loss do not advance. Cause unknown: layer streaming, tokenization/dataloader hang, or my own misconfiguration.	Reproduced; no healthy run obtained. Needs a 20-step run with timestamped step logs.	No. No error was raised; nothing flagged a run that makes no progress.
2. trl / tokenizer / bitsandbytes conflict (observed)	TRL API drift (e.g. tokenizer -> processing_class) can silently drop or misuse the tokenizer, pad/EOS handling or chat template; training still runs on malformed inputs.	Hit these conflicts during setup and fixed them with LLM-assisted debugging; effect on tokenization not verified.	Not verified.
3. System prompt column and prompt format	Columns are mllm_index, system, prompt, chosen, rejected; prompt is a plain string and 459 of 500 rows carry a system prompt in a separate column. As far as I know DPOTrainer reads only prompt/chosen/rejected, so system prompts and the chat template may never reach the model while training still runs.	Column list from Soup inspection; empty-system count (41) from my own count. Not confirmed by decoding a batch.	Not flagged by the inspection.
4. Token caps vs. data length	max_prompt_length=128 and max_length=256 against rows averaging 2,826 chars (max 19,793): long prompts and answers are cut, so the model trains on fragments.	Char lengths from Soup inspection. The share of truncated pairs in tokens was not measured.	Not flagged by the inspection.
5. Style and length bias in the pairs	Empty fields, chosen == rejected, style/length bias, PII, off-topic data.	Counted over all 500 rows: prompt/chosen/rejected empty 0; system empty 41 (all 41 empty fields are system); chosen == rejected 0; exact duplicates 0. Rejected starts with "Konechno" (Sure...) in 239/500 (48%), chosen in 2/500. Mean length chosen 726 vs rejected 1,100 chars (1.5x): DPO can win by dropping that opener and being shorter. Rows are general Q&A/reasoning, not support tickets. PII not checked.	Partly: prints stats, no warning on the opener or length asymmetry.
6. Nothing persisted, no eval split	save_strategy="no" and no eval dataset: the trained adapter is not saved, and no held-out loss or reward accuracy exists, so a good-looking training curve is the only outcome signal.	Save the adapter, reload it, re-run the norm check on the file; add an eval split. Not done.	Not applicable (script bypasses Soup).

4. Verdict: DON'T SHIP

Evidence against: (a) no run through soup train with an advancing step counter, so loss, tok/s and VRAM from it cannot be trusted; (b) peak 6.63 GiB (7.12 GB) vs a 10.97 GB allocated-only budget: unexplained unless gradient checkpointing was silently on (unchecked), since the data implies sequences at the token caps; (c) the only positive evidence is a non-zero LoRA delta norm (1.998), which shows movement, not improvement; (d) the adapter was never saved or evaluated, and several checks were not run (step-0 loss, skipped fp16 steps, BOS/pad handling, token-level truncation). **Must change before shipping:**

- Pin compatible trl / transformers / bitsandbytes versions and find the cause of the stall (my config or a tool bug; report it upstream if the latter). Acceptance: a timestamped 20-step run with step-0 loss 0.693 (proves the reference pass is correct), changing loss and margins, finite grad norm, zero skipped fp16 steps.
- Log per-step max_memory_allocated and token lengths; reconcile with the 10.97 GB budget or explain the difference. Enable gradient checkpointing if sequences grow.
- Run data doctor; verify BOS/EOS/pad handling (pad = eos is set in the script) and prompt format by decoding a batch; measure truncation at 128/256, raise the limits or filter pairs that collapse, decide how to use the system column (41 empty, the rest ignored), and rebalance the "Konechno" opener and the length asymmetry. Save the adapter and add an eval split.
- Held-out evaluation: preference accuracy and reward margin (length-controlled), a general Russian regression check for forgetting, and a shuffled-label control run.
- Change Soup: soup ship should refuse unless it sees step count = expected, step-0 loss near ln2, LoRA delta norm vs. init, and an eval delta.

AI tools usage

I used an LLM for (1) debugging dependency conflicts (TRL API changes, bitsandbytes), (2) writing the weight-verification script, and (3) drafting the structure of this report and the memory arithmetic. I treated its output as untrusted: the norm 1.998 comes from my own run of the script, not from the LLM, and LLM-suggested fixes for TRL API drift are version-specific and can be stale. The arithmetic in section 1 should be independently recomputed, since LLMs are unreliable at long numeric chains.

What surprised me / what still concerns me

I was surprised that soup train could launch, fit in memory and make no progress without any error: a crash would have been better. I was also surprised that the measured VRAM came out well *below* my worst-case estimate; my first instinct was to read that as good news, but I have not explained it yet. I am still concerned that I cannot tell whether the stall is my misuse or a tool defect, that a stalled or degenerate run looks the same as a healthy one without independent checks, and that the only proof I have (a weight norm) is the weakest kind. Attachments (submitted separately): timestamped training logs, raw nvidia-smi output, verify_adapter.py with its output, notebook.